

03 · Einführung in Echtzeitbetriebssysteme

FreeRTOS und Task-Management · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 54

Willkommen in der Profiliga!

- Dritter Termin: vom Dispatcher der Vorlesung 2 zu einem Kernel
- Heute: **ein** FreeRTOS auf dem Cortex-M4 des STM32H755
- Der CM7 übernimmt die im Projekt dokumentierte Startfunktion, keinen zweiten Scheduler
- Ein RTOS strukturiert Tasks; Fristen bleiben nachzuweisen

Notizen

1 / 54

Begriffe aus VL1/VL2: Release, Start, Completion, Deadline, Jitter. Kein SMP über M4 und M7. Keine .ioc im Kursrepo — Versionen am gestellten Cube-Projekt prüfen.

Folie

2 / 54

Agenda für heute

1. Recap: Dispatcher, nicht der rekursive Eigenbau
2. Was ein Kernel leistet — und was nicht
3. Kontext, RAM, Zustände, Prioritäten
4. Ticks, Delays, CMSIS-RTOS2
5. Labor: Heartbeat, HeavyMath, Emergency

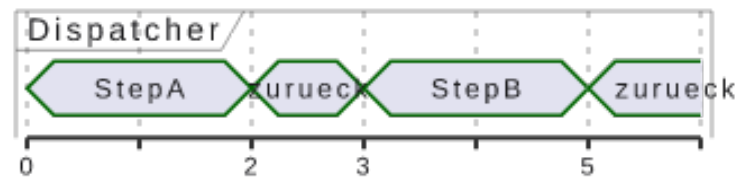
Notizen

2 / 54

Emergency ist heute eine kleine Thread-Flag-Übergabe, keine Queue-Theorie.

Recap: Unser Round-Robin Eigenbau

VL2-Zielbild: zentraler Dispatcher, endliche `step()`-Funktionen, Zustand im TCB.



- Ein Step, der nicht zurückkehrt, blockiert weitere **kooperative** Schritte
- Aktive Interrupts können weiterlaufen
- Ein normaler Funktionsaufruf sichert keine Fortsetzung mitten in einer Endlosschleife

Notizen

3/54

Fehlbeispiel der ersten Laborfassung: `os_yield()` ruft die nächste Task-Funktion auf. Der Stack wächst $A \rightarrow B \rightarrow C$. Das ist kein eigener Task-Stack. Frage: Warum hilft ein privater Stack hier noch nicht, solange wir nur C-Aufrufe haben?

Folie

4/54

Recap: Der Crash im Labor

- Kooperativ: Heavy-Schritt ohne `return` — Heartbeat-Callback kommt nicht dran
- Präemptiv, analog: High bleibt dauerhaft Ready — Low verhungert trotzdem
- Präemption löst das erste Problem und kann das zweite erzeugen
- Harte Fristen brauchen Grenzen der Arbeit, nicht nur einen Kernel

```
/* Fehlbeispiel, historisch: nicht der VL2-Dispatcher */
void Task_1(void) {
    while (1) { LED_On(); os_yield(); /* ruft andere Task */ }
}
```

Notizen

4/54

„Harte Echtzeit braucht immer Präemption“ ist zu pauschal. Begrenzte kooperative Schritte können Fristen erfüllen. Bare-Metal kann per IRQ schlafen.

Die Notwendigkeit eines echten RTOS

- Viele Aufgaben, Fristen und Wartepunkte werden sonst zur Flag-Suppe
- Der Kernel bietet Scheduling, Objektallokation, Kommunikation, Synchronisation, Port
- Mutex ist keine allgemeine ISR-Lösung; aus der ISR nur dafür vorgesehene Übergaben
- Zertifizierung und Speicherschutz entstehen nicht durch „FreeRTOS einschalten“

Queues und Mutex vertiefen wir später. Heute reicht die Einordnung.

Was leistet ein RTOS?

Baustein	Rolle
<code>tasks.c</code>	Zustände, Ready-Listen, Delays
<code>list.c</code>	Listen für diese Objekte
<code>queue.c</code>	Queues (später)
<code>heap_x.c</code>	eine mögliche Allokation
CM4F-Port	C und Assembler: PendSV, SVC, Tick

- Plattformunabhängig ist die API-Idee, nicht Prioritätszahl, Takt oder Treiber

Der Port ist nicht „alles Assembler“. HAL-Treiber sind eine Nachbarschicht, nicht der Port.

Der Markt für Echtzeitbetriebssysteme

- Kommerziell u. a. VxWorks, QNX; im Auto oft AUTOSAR-OS
- Offen u. a. Zephyr und FreeRTOS
- „Marktführer“ oder „safety-zertifiziert“ nur mit benannter Quelle und Konfiguration
- Wir nutzen FreeRTOS, weil Cube es für den STM32 anbindet — nicht als Beweis einer Frist

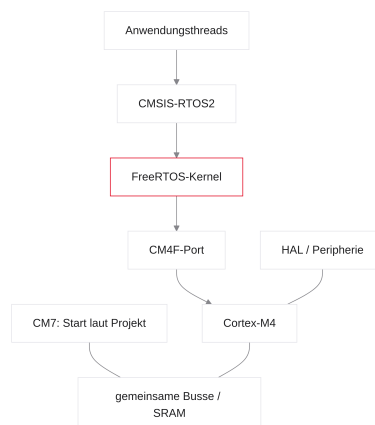
Footprint-Werbeahlen (wenige KB) hängen von Optionen ab. Für unser Projekt zählt die Map-Datei, nicht ein Datenblatt-Slogan.

Lernziele des heutigen Tages

1. Dispatcher-Callback und RTOS-Thread unterscheiden
2. Stack, TCB und übriges RAM grob bilanzieren
3. Ready, Running und Blocked an einer CPU-Spur erklären
4. Tickphase, `osDelay` und `osDelayUntil` auseinanderhalten
5. Emergency per Thread-Flag aus einer erlaubten ISR wecken

Am Ende keine Behauptung „Jitter ungefähr null“.

Architektur: Bare-Metal vs. RTOS



- Ein Kernel auf dem CM4; Tasklisten wandern nicht zum M7
- Zwei Kernel später wären eine eigene AMP-Entscheidung
- Ohne MPU-Port isoliert der Stack nur organisatorisch

Eigener NVIC und eigene Vector Table je Kern, wie in VL1. Gemeinsamer Speicher bleibt Interferenz.

Das Kernparadigma: Latenz vs. Determinismus

Synthetische Antwortzeiten, Deadline $D = 10$ ms:

- Folge 2/4/6/8 ms: Schwankung, aber kein Miss
- Ein Lauf mit $R = 12$ ms: Miss
- Hohe Priorität ist keine Obergrenze
- Konstante Zusatzlatenz ist kein Jitter

Bestandteile von R , ohne hier die RTA zu wiederholen: eigener CPU-Bedarf, höherprioriäre Arbeit, IRQs, Blocking, Dispatcher.

Notizen

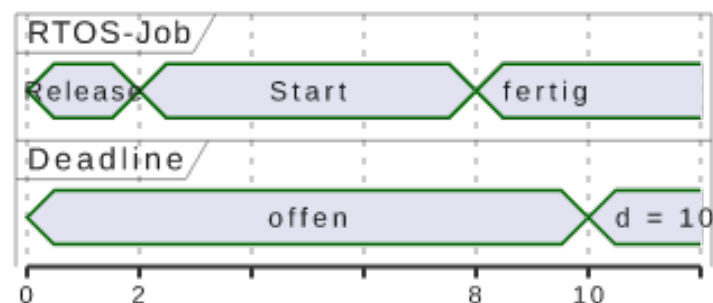
10/54

Frage: Welche Annahmen fehlen für eine harte Garantie? Antwort: beschränkte C , Prioritäten, modellierte IRQ- und Sperrzeiten, kein unbegrenztes Blocking, gültiger Test. Wenige Kurven beweisen das nicht.

Folie

11/54

Visualisierung: Jitter-Vergleich



- Die Skizze zeigt einen gehaltenen Job mit $R = 8$ ms
- Eine Bare-Metal-ISR kann dieselbe Frist mit weniger Dispatcheranteil halten
- Der Vergleich ist nicht „Super-Loop schlecht, RTOS gut“

Notizen

11/54

Lehrwerte, keine Boardmessung. Startlatenz, Antwortzeit und deren Schwankung getrennt nennen.

Einführung in FreeRTOS

- Richard Barry, später Amazon/AWS; Lizenz MIT
- Portierbar, deshalb der STM32-Cube-Standard in diesem Kurs
- Konkrete ROM-/RAM-Größe: Linker-Map der **verwendeten** `FreeRTOSConfig.h`
- Version von Kernel, CMSIS-Wrapper und CubeH7 am Projekt ablesen

Notizen

12 / 54

Öffentliches `GitHub-main` ist nicht die Version im Labor.

Folie

13 / 54

Die FreeRTOS Architektur

- Anwendung → CMSIS-RTOS2 → Kernel → Port → Cortex-M4
- `tasks.c`, `list.c`, `queue.c`, `heap_4.c` o. ä. nach Rolle
- PendSV/SVC im Port; SysTick ist eine übliche Tickquelle, keine Pflicht
- CMSIS macht nicht jeden Backendwechsel ohne Anpassungen möglich

Notizen

13 / 54

Prioritätsabbildung, Heap und Timer bleiben plattformabhängig.

Folie

14 / 54

Zusammenfassung: Warum ein RTOS?

- Struktur: Threads statt einer Flag-Suppe
- Kosten: Ticks, Wechsel, Stacks, Fehlersuche
- Keine automatische Pünktlichkeit und kein automatischer Speicherschutz
- Präemption ersetzt kooperative Rückkehrpflicht, nicht die Analyse

Notizen

14 / 54

„Geschwindigkeit gegen garantierte Pünktlichkeit“ ist die falsche Tauschformel.

Das FreeRTOS Task-Modell

- Native Task-Funktion kehrt nicht normal zurück
- Ende über `vTaskDelete(NULL);` CMSIS: `osThreadExit()`
- Ein bloßes `return` führt portabhängig oft in eine Assert- oder Fehlerschleife, nicht zwingend in den HardFault
- Labor-Threads: `for (;;)` und RTOS-Warten

```
void Heartbeat(void *arg) {  
    (void) arg;  
    for (;;) {  
        /* Arbeit, dann osDelay / osDelayUntil */  
    }  
}
```

Notizen

15/54

Der gespeicherte Kontext liegt auf dem Task-Stack. Der TCB hält insbesondere den Stackpointer, nicht den PC als loses Feld.

Folie

16/54

Der Task Control Block (TCB) in FreeRTOS

- `pxTopOfStack`: Oberkante des gesicherten Kontexts
- Listenelemente: Ready, Delayed, ...
- `uxPriority`: nativ `UBaseType_t`, nicht pauschal `uint8_t`
- CMSIS sieht `osPriority_t`; der Wrapper bildet ab und kann Werte ablehnen

Notizen

16/54

Drei TCBs, drei Stacks. PSP für den laufenden Thread, MSP für Handler. IRQ-Stack extra bilanzieren.

Speichermanagement: Der Stack

- Lokale Variablen und Aufrufe liegen auf dem **Task-Stack**
- Hardware-Frame: R0–R3, R12, LR, PC, xPSR; Software zusätzlich R4–R11, portabhängig FPU
- 128 Einträge `StackType_t` × 4 Byte = **512 Byte**
- CMSIS `stack_size` = 512 bedeutet **512 Byte**, nicht 512 Wörter
- Cube-Feld gegen den generierten Wert prüfen

Notizen

17 / 54

Statischer Puffer reserviert früh, beweist aber keine ausreichende Tiefe.

Folie

18 / 54

Berechnung des RAM-Bedarfs

Annahme, ausdrücklich nicht die Map-Datei: TCB ≈ 80 Byte.

Position	Byte
Stack 512 + TCB 80	592
Drei Threads 512 / 1024 / 1024 + 3×80	2800

Dazu, oft **andere** Pools: Idle, Timer-Task, Queues, Heap-Metadaten, Alignment, MSP.

- Das gesamte H755-SRAM ist kein frei zusammenhängender CM4-Heap
- Dynamische Anlage kann scheitern; statische Reservierung verlagert die Planung nach vorn

Notizen

18 / 54

2800 Byte decken nur diese drei Beispieltasks unter der 80-Byte-Annahme. Linker-Map und `configTOTAL_HEAP_SIZE` liefern den Projektwert.

Speicherallokation: Statisch vs. Dynamisch

- Dynamisch: `osThreadNew` kann NULL liefern
- Statisch: Speicher ist vorher da, Stacktiefe bleibt eine Messfrage
- Große Puffer brauchen Eigentümer und Lebensdauer
- Eine globale Variable ist keine automatische Safety-Lösung
- MPU-Isolation nur mit dem dafür gebauten Port und passender Konfiguration

Notizen

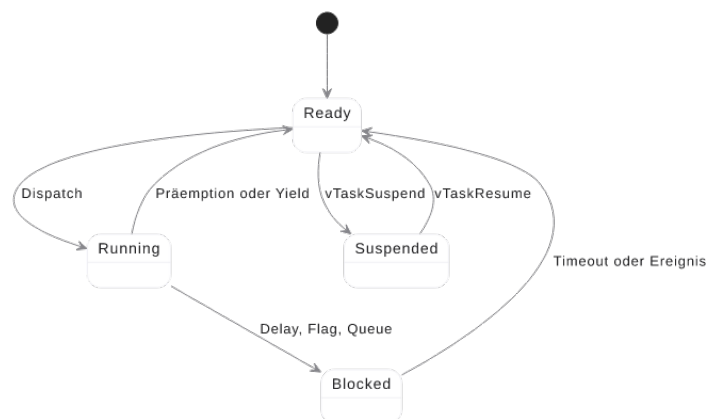
19/54

Heap voll $\neq$ HardFault. Zuerst Handle prüfen.

Folie

20/54

Das FreeRTOS Zustandsmodell



- Suspend ist administrativ, nicht dasselbe wie Warten auf ein Ereignis
- CMSIS `osThreadState_t` hat **kein** eigenes Suspended; die Abfrage zeigt das oft als Blocked
- Blocked: der Task führt keine eigenen Befehle aus; Tick und Wakeup kosten trotzdem Zeit

Notizen

20/54

Frage: Ist Ready schon Running? Nein. Es darf höchstens ein Thread rechnen; IRQ/Kernel extra.

Der „Running“-Zustand

- Genau ein Thread hat die CPU
- Idle rechnet, wenn niemand sonst Ready ist
- `HAL_Delay` ist in der üblichen HAL ein Busy-Wait: Zustand bleibt Running, höherpriorere Threads können trotzdem präemptieren
- Busy-Wait ist Last, kein automatisches „Echtzeit zerstört“ — im Labor trotzdem `osDelay`

Notizen

21 / 54

Nicht behaupten, `HAL_Delay` laufe ununterbrochen allein bis zum Ende.

Folie

22 / 54

Der „Ready“-Zustand

- Will rechnen, wartet auf Zuteilung
- Nach Timeout kann ein höherpriorer Thread nach IRQ-Ende und `PendSV` **sofort** laufen
- Ready heißt nicht „noch einen ganzen Tick warten“
- Maskierung oder eine laufende höhere ISR verzögern den Wechsel

Notizen

22 / 54

Das Intervall zwischen Timeout und Lauf ist Dispatcherzeit, keine erfundene 1-ms-Wechseldauer.

Folie

23 / 54

Der „Blocked“-Zustand (Die Magie des RTOS)

- Eigene Auslastung des wartenden Threads: keine Instruktionen
- Gesamt-CPU kann durch `HeavyMath` trotzdem voll sein
- `osDelay / osThreadFlagsWait` blockieren; `HAL_Delay` nicht
- Aus der ISR kein `osDelay` und kein Mutex-Warten

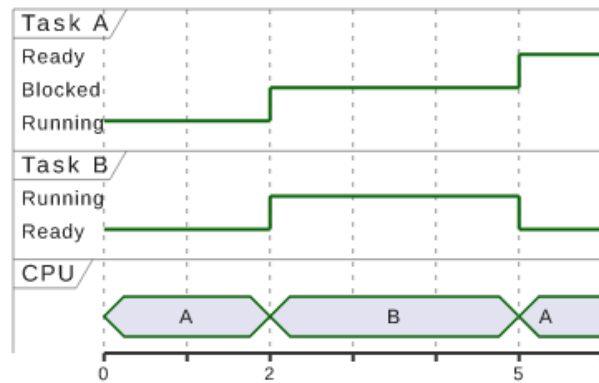
Notizen

23 / 54

„0 % eigener Code“ und „0 % Kernel“ nicht vermischen.

Visualisierung: Task-Zustände im Zeitverlauf

A höher als B. A läuft, ruft Delay, B läuft, A wird Ready und läuft nach zulässigem Wechsel wieder.



- Bei $t = 5$ rechnet A, nicht „A wartet noch bis $t = 6$ “
- Kurze PendSV-Zeit im Detail denken, nicht als 1-ms-Balken messen

Wenn die Zeichnung A zwischen 5 und 6 nur Ready zeigte, wäre das ein Fehler ohne benannte Ursache.

FreeRTOS Scheduler-Konfiguration

- Höchste nichtleere Ready-Priorität gewinnt
- Suche oder optimierte Bitmap: abhängig von `configUSE_PORT_OPTIMISED_TASK_SELECT`
- Deshalb keine universelle Taktzahl für „den“ Scheduler
- `configMAX_PRIORITIES` und der CMSIS-Wrapper gemeinsam prüfen

Zu wenige Stufen: der Wrapper klemmt oder weist eine CMSIS-Priorität zurück.

Prioritäten in FreeRTOS (Achtung: Falle!)

Skala	Richtung
Native FreeRTOS-Task	größere Zahl = dringender
CMSIS <code>osPriority_t</code>	ebenso, andere Symbole
NVIC, logisch	kleinere Zahl = dringendere IRQ

Labor-Ordnung: Emergency > Heartbeat > HeavyMath > Idle.

- Konkrete Abbildung steht in der verwendeten `cmsis_os2.c`
- „Oft 7 Stufen“ nicht mit einem Wrapper mischen, der mehr CMSIS-Stufen erwartet
- `osThreadYield`: hohe Priorität bleibt Ready und wird wieder gewählt

Notizen

26 / 54

Frage: Warum hilft Yield dem niedrigpriorären Task nicht? Weil die hohe Aufgabe weiter bereit ist. Gleichstand plus `configUSE_TIME_SLICING` ist ein eigener Fall.

Folie

27 / 54

Die Idle-Task (Prio 0)

- Idle hat die niedrigste native Priorität, muss sie aber nicht allein besitzen
- Dauerhaft bereites HeavyMath **oberhalb** Idle verhindert Idle-Lauf, Cleanup und Idle-Schlaf
- Hooks und das Löschen beendeter Tasks sind konfigurationsabhängig
- `WFI` ist kein automatischer Standardpfad jeder `FreeRTOSConfig.h`

Notizen

27 / 54

Frage: Warum läuft Idle im normalen Labor nicht? Weil HeavyMath bereit bleibt und höher ist.

Strom sparen: Tickless Idle

- Tickless unterdrückt geeignete Kernel-Ticks
- Der übliche CM4F-Port verlängert SysTick; das ist kein eigener Low-Power-Timer
- LPTIM/RTC brauchen eine passende Portierung
- 24-Bit-Zähler: bei 100 MHz etwa 167,8 ms, bei 240 MHz etwa 69,9 ms
- „500 ms mit unverändertem SysTick“ nicht zeichnen
- Ein weiterlaufender HAL-TIM6 kann trotzdem wecken
- Deep Sleep braucht PWR, Clock und beide Kerne — das HeavyMath-Labor ist kein Tickless-Versuch

Notizen

28 / 54

Tickless erzeugt weder automatisch Deep Sleep noch einen korrigierten Betrieb ohne `vTaskStepTick`.

Folie

29 / 54

Der RTOS-Herzschlag: Die Tick-Rate

- `configTICK_RATE_HZ` setzt die **Kernel**-Tickperiode, nicht die Wanduhr allein
- `osDelay(1)` endet an einer Tickgrenze
- Aufruf bei 0,1 ms: Ready bei 1 ms (Dauer 0,9 ms)
- Aufruf bei 0,9 ms: Ready bei 1 ms (Dauer 0,1 ms)
- Danach kann höherpriorie Arbeit den echten Start noch schieben

Notizen

29 / 54

Frage: Garantiert ein Tick mindestens 1 ms? Nein. Bei 1 kHz ist die Periode 1 ms, die Restzeit bis zur Grenze kann kürzer sein.

Zeit-Umrechnung (Die goldene Regel)

Aufrundender Helfer, nicht `pdMS_TO_TICKS` ungeprüft:

```
uint32_t ms_to_ticks_ceil(uint32_t ms) {
    uint32_t f = osKernelGetTickFreq();
    if (ms == 0 || f == 0) return 0;
    return (uint32_t)((uint64_t)ms * f + 999u) / 1000u;
}
```

ms	1000 Hz	100 Hz, ceil
1	1	1
5	5	1
100	100	10
500	500	50

- Ceil vermeidet 0 Ticks für kleine positive ms, garantiert aber keine Mindest-Wartzeit
- `pdMS_TO_TICKS` rundet je nach Makro ab: 1 ms oder 5 ms bei 100 Hz können 0 sein
- `vTaskDelay(0)` yieldet; `osDelay(0)` ist nicht dieselbe Warte-API

Notizen

30 / 54

Null, Überlauf der 32-Bit-Multiplikation und die Tickfrequenz prüfen. Lehrfrequenzen, nicht als gemessenen H755-Takt ausgeben.

Folie

31 / 54

Der Context Switch (Overhead-Analyse)

- Lehrannahme 150–300 Zyklen, **kein** Messwert
- Bei 100 MHz: 1,5–3 μ s
- 200 Zyklen, 500 Wechsel/s, 100 MHz: 0,1 % CPU, Tick und ISRs extra
- Hardware-Frame, Software-Register, PSP setzen, `EXC_RETURN`
- FPU nur wenn der Port den Kontext mitsichert

Notizen

31 / 54

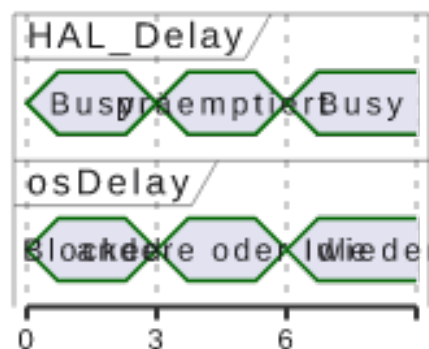
Kein eigener Assembler im Labor. Referenz bleibt der CM4F-Port der eingesetzten Version.

Wann wechselt der Scheduler?

- Tick, Yield, Blockieren, oder eine ISR, die einen höheren Thread bereit macht
- Der Wechsel läuft in PendSV, wenn die IRQ-Priorität das zulässt
- Nicht jeder Wechsel wartet auf den nächsten Tick
- SysTick und PendSV im üblichen Port: niedrigste IRQ-Priorität (VL2)

Pending ist nicht Running.

vTaskDelay vs. HAL_Delay



- Busy-Wait: eigene Last, Zustand Running, präemptierbar
- osDelay: Blocked, CPU für Ready oder Idle
- Laborregel für diese Threads: CMSIS-Warten

Heartbeat unter HeavyMath bleibt lange Blocked, die CPU-Summe bleibt hoch. Das ist kein Widerspruch.

Drift-freies Timing: vTaskDelayUntil

Ideales Modell, Tick 1 ms, Störungen 0. Arbeit 2/4/1 ms, danach relatives Delay 10 Ticks:

- Starts 0/12/26/37 ms
- Absolutes Sollraster 0/10/20/30 ms; die Arbeiten passen hier vor den nächsten Punkt
- Tatsächliche Starts können 0/10,4/20,2/30,6 ms sein: Raster gleich, Zuteilung schwankt

```
uint32_t next = osKernelGetTickCount();
for (;;) {
    do_work();
    next += period_ticks;           /* Wrap in uint32_t */
    osStatus_t st = osDelayUntil(next);
    if (st == osErrorParameter) { /* Ziel zu weit oder ungültig */
    }
```

- osDelayUntil erwartet den **absoluten** Tick; native xTaskDelayUntil hat eine andere Signatur
- Überlast: Miss zählen und neu synchronisieren oder die Aktivierung verwerfen — keine stille Aufholschleife
- Grenze der Until-API: Abstand höchstens $2^{31} - 1$ Ticks; ein kleinerer Zähler kann nach Wrap in der Zukunft liegen

„Startet exakt alle 100 ms“ ersetzen durch: nächstes Freigabeziel. Der CPU-Start hängt vom Scheduler ab.

Die Datei FreeRTOSConfig.h

- Tickrate, Heap, maximale Prioritäten, Assert, Stack-Overflow-Check
- configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY: logische API-Schwelle
- configMAX_SYSCALL_INTERRUPT_PRIORITY: derselbe Wert in die Registerbits geschoben
- Grouping und __NVIC_PRIO_BITS am Projekt prüfen; 4 Bit und Schwelle 5 sind nur ein Beispiel
- Dann dürfen logische Stufen 0–4 keine Kernel-API rufen; 5–15 nur die erlaubten FromISR-Operationen

Nicht die logische 5 in das NVIC-Register schreiben, als wäre sie schon verschoben.

Zusammenfassung: Scheduler und Timing

- Auswahl: höchste bereite Taskpriorität, Kosten konfigurationsabhängig
- Tickphase erklärt kurze `osDelay(1)`-Reste
- Until hält das Sollraster, nicht jeden Startzeitpunkt
- Idle arbeitet nur, wenn niemand darüber dauerhaft Ready ist

Frage: Warum kann `osKernelStart` zurückkehren? Weil die Initialisierung fehlschlagen kann. Bei Erfolg kehrt der Aufruf normalerweise nicht zurück.

FreeRTOS API vs. CMSIS-RTOS v2

- CMSIS-RTOS2 ist der Adapter, den Cube für STM32 erzeugt
- Weniger direkte Abhängigkeit von `vTask...`, aber nicht jedes RTOS bietet dieselben Objekte
- Thread-Flags: ein Bit signalisiert; wiederholtes Setzen **zählt nicht** verlustfrei
- Rückgabe von `osThreadFlagsWait` sind Flag-Bits, Fehler über gesetzte Fehlerbits — nicht mit `osOK` vergleichen

`osThreadFlagsSet` darf aus einer ISR, `osThreadFlagsWait` nicht.

STMicroelectronics und CMSIS

- Heute: Kernel auf dem CM4, CMSIS_V2
- Ab CubeIDE 2.0 oft: CubeMX separat, Projekt importieren
- Ältere IDE: Device Configuration Tool innen
- Menüpunkte an der **installierten** Version prüfen, nicht auswendig
- Generierte Thread-Funktionen können in `main.c` oder `freertos.c` liegen

Notizen

38/54

Keine .ioc im Repo, deshalb keine behauptete Cube-Version.

Folie

39/54

Ein CMSIS-Thread im Code

```
osThreadId_t hHb;
const osThreadAttr_t attrHb = {
    .name = "Heartbeat",
    .priority = osPriorityNormal,
    .stack_size = 512    /* Byte, generierten Wert gegenprüfen */
};

void app_create(void) {
    if (osKernelInitialize() != osOK) { Error_Handler(); }
    hHb = osThreadNew(Heartbeat, NULL, &attrHb);
    if (hHb == NULL) { Error_Handler(); }
    if (osKernelStart() != osOK) { Error_Handler(); }
    /* Erfolg: hier kommt man üblicherweise nicht an */
}
```

Notizen

39/54

Jedes Handle und jeder Status. Dieselbe Prüfung für HeavyMath und Emergency.

Starten des Betriebssystems

1. Takt, GPIO, EXTI — IRQ des Tasters noch aus
 2. `osKernelInitialize` erfolgreich
 3. Threads anlegen, Handles prüfen
 4. `osKernelStart`
 5. Erst dann EXTI freigeben, wenn das Emergency-Handle gilt
- Tickfrequenz kommt aus der Config, nicht automatisch aus „immer 1 ms“.

Notizen

40 / 54

Fehlerzweige nicht löschen, nur weil der Erfolgspfad nicht zurückkehrt.

Folie

41 / 54

Fehlerbehandlung: Wenn RAM fehlt

- `osThreadNew` → NULL: Heap, Stackgröße oder Priorität
- Assert, Overflow-Hook, High-Water-Mark in der Einheit der API
- Heap-Reste und Linker-Map, nicht „Stack verdoppeln“
- HardFault hat viele Ursachen; Overflow beschädigt zuerst Speicher und faultet nicht zuverlässig

Notizen

41 / 54

Keine 90%-Behauptung. Endliche Watermark-Tests sind Stichproben.

Folie

42 / 54

Schritt 1: STM32CubeIDE-Setup

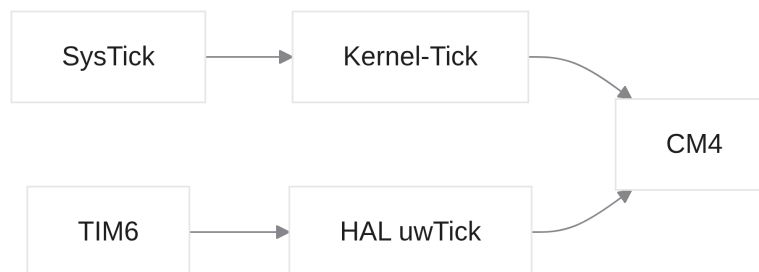
- Board NUCLEO-H755ZI-Q, MCU STM32H755ZIT6, FreeRTOS nur für den CM4, Interface CMSIS_V2
- Heap so groß wählen, dass die drei Threads plus Idle/Timer laut Map passen — 16 KB ist ein Startwert, kein Beweis
- LED LD2 = PE1 (MB1363); Tasterpin und EXTI am Schaltplan prüfen
- Prioritäten anlegen und in `cmsis_os2.c` nachsehen, ob sie unverändert ankommen

Starvation-Versuch später: HeavyMath auf `osPriorityAboveNormal`, weiter unter Emergency. Bleiben beide auf High, Zeitscheibe und Gleichstand extra betrachten.

Schritt 2: Der Konflikt der Timer

Gewählte Laborkonfiguration, kein Naturgesetz:

- SysTick: Kernel-Tick
- TIM6, falls frei: HAL `uwTick`
- Beide auf dem CM4, Priorität und Handlerkette prüfen
- Ein gemeinsamer SysTick ist bei korrekter Integration möglich
- FreeRTOS kann auch einen anderen Tick-Timer nutzen



„Gleicher SysTick stürzt immer ab“ streichen. Getrennte Basen sind die konkrete Laborwahl, damit HAL-Delay und Kernel sich nicht dieselbe ISR teilen müssen.

Schritt 3: Task-Design für das Labor

Thread	Priorität	Verhalten
Emergency	hoch	wartet auf Flag, kurze Reaktion
Heartbeat	normal	Sollraster 500 ms, Toggle PE1
HeavyMath	darunter	dauerhaft Ready, Ergebnis benutzen
Idle	nativ 0	kommt im Normalversuch nicht dran

- Vollständiger LED-Zyklus bei Toggle alle 500 ms: etwa 1 s
- Emergency keine Endlos-Ready-Schleife und kein `printf` als Messmarker

Überblick 0–1100 ms, Lehre: Heartbeat-Ziele 0/500/1000, Ereignis bei 750, HeavyMath in den Lücken.
Kurze Kernelzeiten nur im Detail, nicht als gemessene Balken.

Code-Struktur nach Generierung

Code in den Funktions-USER CODE-Block, nicht in Header_...-Kommentare. Keine zweite Definition derselben Thread-Funktion.

```
/* USER CODE BEGIN StartHeartbeatTask */
uint32_t next = osKernelGetTickCount();
uint32_t period = ms_to_ticks_ceil(500);
for (;;) {
    HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
    next += period;
    if (osDelayUntil(next) == osErrorParameter) {
        next = osKernelGetTickCount(); /* bewusst neu aufsetzen */
    }
}
/* USER CODE END StartHeartbeatTask */
```

Bei verpasstem Ziel nicht in einer Aufholschleife stecken bleiben. `osError` vom Kernelstart nicht mit einem gewöhnlichen Miss verwechseln.

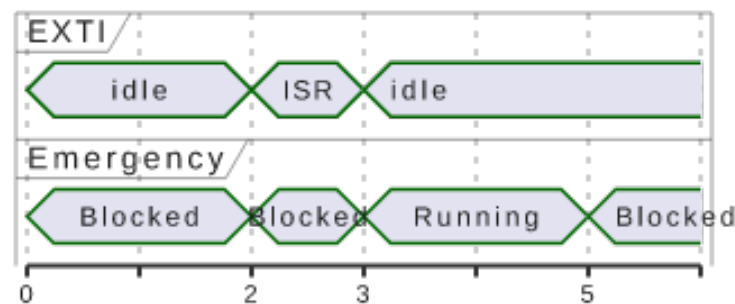
Code-Skelett: Die HeavyMath-Task

```
/* USER CODE BEGIN StartHeavyMathTask */
static volatile uint32_t acc;
for (;;) {
    acc = acc * 1664525u + 1013904223u; /* benutztes Ergebnis */
}
/* USER CODE END StartHeavyMathTask */
```

- Kein `osDelay`, keine IRQ-Sperre
- Der seltene GPIO-Schreibzugriff hält die Rechnung lebendig, ohne die LED-Messung zu dominieren

Ein ungenutzter Zähler wird bei Optimierung oft entfernt. Priorität unter Heartbeat, sonst Starvation.

Vom Taster zum Emergency-Thread



- ISR: `osThreadFlagsSet (hEm, 0x1)` — nur wenn Handle und Kernel leben und die IRQ-Priorität es darf
- Thread: `osThreadFlagsWait (0x1, osFlagsWaitAny, osWaitForever)`, dann kurze Reaktion, wieder warten
- Dasselbe Bit kann mehrere Flanken zu einem Wake zusammenfassen
- Entprellung und erlaubte Rate festlegen; der Thread wartet nicht auf den nächsten Tick

Native `xTaskNotifyFromISR` plus `portYIELD_FROM_ISR` nur als Vergleich. Vorhandenen HAL-Callback nutzen, keinen zweiten Handler erzeugen.

Beobachtung 1: Der Präemptions-Beweis

- HeavyMath bleibt Ready und rechnet in den Lücken
- Heartbeat wird am Sollpunkt Ready und verdrängt HeavyMath, weil die Priorität höher ist
- Das zeigt Präemption, nicht eine harte Frist und nicht „Jitter ≈ 0 “
- Stabiles Blinken ist kein Deadline-Nachweis

Messung, falls ein Oszi da ist: Pulsbreite und Startabstand getrennt. Lehrkurven synthetisch lassen. Ohne Board keine Messwerte eintragen.

Visuelle Erklärung: Präemption im Labor

Ziele Heartbeat: 0, 500, 1000 ms. Angenommenes Flag bei 750 ms.

- Dazwischen HeavyMath, solange Emergency blockiert
- Bei 750: EXTI, Flag, PendSV, kurze Emergency-Arbeit, wieder Blocked
- Zwei Spuren denkbar: Stimulus und Emergency-GPIO
- Endpunkte der Latenz vorher festlegen (Flanke bis erste Thread-Anweisung oder bis GPIO)

Notizen

49 / 54

IRQ- und Kernelanteile nicht in die Task-Balken schummeln. 150–300 Zyklen bleiben Lehrannahme.

Folie

50 / 54

Beobachtung 2: Starvation durch falsche Prioritäten

- HeavyMath auf `osPriorityAboveNormal`: über Heartbeat, unter Emergency
- Heartbeat wird Ready und bleibt es; die LED kann an **oder** aus stehen bleiben
- Das ist Starvation, keine Priority Inversion
- `osThreadYield` in HeavyMath ändert nichts, solange HeavyMath Ready bleibt
- Danach die normalen Prioritäten wiederherstellen

Notizen

50 / 54

Priority Inversion braucht eine gemeinsam gehaltene Ressource. Die kommt in einem späteren Termin.

Folie

51 / 54

Debugging: Stack Overflow erkennen

1. Kernelstart oder Handle fehlgeschlagen?
2. Takt, IRQ, Boot, EXTI-Priorität?
3. Task Ready, aber verdrängt?
4. Assert oder Overflow-Hook?
5. Watermark, Heap, Map, Fehlerregister?
 - Hook und `configASSERT` einschalten, wenn das Projekt das vorsieht
 - Keine CMSIS-Funktion erfinden, die es nicht gibt; Watermark ist backendbezogen (`uxTaskGetStackHighWaterMark` o. ä.)
 - Reserve aus Messung, nicht aus Verdopplung als einzigem Rezept

Frage: Warum ist „90 % sind Stack Overflow“ falsch? Weil auch Priorität, Heap, IRQ-Schwelle und Boot dieselbe LED dunkel lassen.

Zusammenfassung des Tages

- Kernel auf dem CM4 strukturiert Threads; Analyse bleibt nötig
- 512 Byte Stack $\neq$ 128 Byte; drei Beispielthreads unter der 80-Byte-TCB-Annahme: 2800 Byte
- Blocked spart eigene Befehle, nicht jeden Kernel-Tick
- `osDelayUntil` setzt das Raster; Starts können trotzdem zittern
- Emergency: Flag aus erlaubter ISR, dann kurze Arbeit

Kontrollen: $128 \times 4 = 512$; $512 + 80 = 592$; $512 + 1024 + 1024 + 240 = 2800$. Relative Starts 0/12/26/37. Absolut 0/10/20/30. 150–300 Zyklen bei 100 MHz sind 1,5–3 μ s.

Ausblick auf Termin 4

- Gemeinsame Daten, Mutex, Priority Inversion
- Bis dahin: keine blockierenden API-Aufrufe in ISRs
- Laborprojekt mit den drei Threads mitbringen

Kommunikation nicht heute vorwegnehmen. Thread-Flags bleiben die kleine Ausnahme für Emergency.

Quellen

- Arm CMSIS-RTOS2: Kernel, Threads, Wait, Thread Flags
- ARM-software/CMSIS-FreeRTOS, Stand am Wrapper prüfen
- FreeRTOS-Kernel, Port GCC/ARM_CM4F; Dokument zu Delays, Stack, Tickless, Cortex-M-Prioritäten
- ST AN5361, PM0214, UM2408, RM0399 — Revision am verwendeten PDF

Notizen

Öffentliche main-Zweige belegen nicht die Laborversion. Keine Oszi-Zahlen ohne Protokoll.